

# HRISHIKESH PINGLE

📞 +91 9137985297 — ✉️ hppingle — 🌐 linkedin.com/in/hppingle — 🎧 Blaeriz

## Summary

I work on quant research tooling, time tools, execution and matching engines, data pipelines on Rust and C++. I also have hands on experience with NTP, PTP, WhiteRabbit protocols and their applications in pico second accuracy synchronization networks. Some of the places I have executed projects: BANK OF BARODA, BARC

## Skills

|                                        |                                                                          |
|----------------------------------------|--------------------------------------------------------------------------|
| <b>Languages</b> C, C++, Rust          | <b>Hardware</b> STM32, ESP32, Arduino, RaspberryPI                       |
| <b>Frameworks</b> Ratatui              | <b>VCS</b> Git, JJ                                                       |
| <b>Databases</b> KDB+/Q, MySQL, DuckDB | <b>Performance Measurement</b> Hyperfine, Dhat, Flamegraph, Oha, Tracing |

## Experience

### Omnibook (Raeth Corporation Inc.)

Sept 2026 – Present

*Low Latency Exchange Developer*

- Built a matching engine raft consensus cluster with 300k orders/s and microsecond p99.
- Designed the data-packet frames and payloads for communication between different microservices over ZeroMQ.
- Built a local OMS rolling WAL (Walk ahead log) for durability, recovery and persistence acknowledgment.
- Worked on designing a complete Broker + Exchange architecture with monotonically increasing sequence events.
- Work on previous designs of the WAL to change it from synchronous to asynchronous while minimizing risks using failover systems.

### WINspect Technologies Pvt. Ltd.

Jan 2026 – Present

*Systems Software Developer*

- Engineered a cross-platform rust tool to view a complete diagnostic summary of your GNSS/WhiteRabbit nodes' fleet across a distributed, precision-timing infrastructure.
- Built a secure device communication and configuration pipeline supporting fleet monitoring, provisioning, and fault diagnostics across White Rabbit and PTP-synchronized nodes.
- Used performance measuring tools like Hyperfine, Flamegraphs etc. to identify bottlenecks and increase performance
- Built custom, high-performant, SNMP data parser for decoding messages from NTP devices.
- Performed a complete pico second standard deviation test between Grandmaster, Master and WhiteRabbit Nodes using fiber and Oscilloscope.
- Executed end-to-end design, installation, configuration, deployment and testing of NTP and WhiteRabbit time synchronization architecture of the 2nd Largest public sector bank of India (BOB)

## Projects

### Algorithmic Trading and Research Engine (Arrow) - Github

(Discontinued) Personal

- Built an end-to-end equities trading engine in Rust ( 2.7K LOC) spanning binary market-data ingestion, a columnar historical store, event-driven backtesting, and live paper trading, exposed through a six-command clap CLI over tokio async runtime.
- Implemented a zero-copy parser for the broker's binary WebSocket protocol handling zstd-compressed batches of heterogeneous, self-delimiting frames. Handles all types of possible input frames.
- Designed a single Strategy trait driving backtest, paper, and live execution from one implementation, and an OrderExecutor trait isolating fill simulation from broker order placement. Adding a strategy requires no changes to any engine.
- Engineered a resilient historical ingestion pipeline: timeframe-aware request chunking (7/30/365-day windows), exponential-backoff retry on HTTP 429, boundary-overlap deduplication, and serialization into Apache Arrow RecordBatch → Parquet with an explicit typed schema, giving a local columnar cache that backtests load without re-hitting the API.
- Wrote a backtest engine computing weighted-average entry basis, per-leg realized PnL, full equity curve, peak-to-trough max drawdown, and win-rate statistics, with results serialized to JSON; live mode aggregates raw ticks into IST-bucketed OHLCV candles and shuts down cleanly on SIGINT via tokio::select!.
- Added symbol resolution with a TOML override layer over a 24-hour-cached instruments API, and property-style unit tests covering frame splitting and truncated-payload rejection.

### Limit Order Book Simulator - GitHub

Personal

- Engineered a high-performance **limit order book engine in C** implementing price-time priority, scaling to **1M ticks, 25M+ order operations, and 1M+ trades** with **100 heterogeneous agents** (70 noise, 5 market makers, 25 informed) on a single core.

- Achieved **sub-100ns p50 latency** for add/cancel/match operations (match p50  $\sim$  15ns) and **255K+ ticks/sec** throughput via cache-efficient data structures and profiling-driven optimization.
- Optimized order cancellation from  **$O(n)$**   $\rightarrow$   **$O(1)$**  using doubly-linked lists and direct node indexing, reducing median cancel latency from microseconds to nanoseconds and enabling deep-book stress tests (**300K+ orders per level**).
- Implemented production-style structures: **red-black tree** for  $O(\log n)$  price levels, **hash map** for  $O(1)$  order lookup, and FIFO queues per level; built-in nanosecond benchmarking with p50/p99 tracking.
- Built a **kdb+ persistence layer in raw C** (KxSystems C API) with async batched I/O (1K orders/flush), agent-tagged orders, and global order IDs; visualized mid-price evolution across **1M ticks** via a Python analytics pipeline.
- The pipeline is stable, does not crash and is deterministically reproducible.

#### Lock-Free SPSC Order Generator - GitHub

Personal

- Developed a lock-free, cache-aligned **SPSC ring buffer in C++17** using atomic operations and power-of-two indexing.
- Benchmarked at **72M orders/sec** and **sub-100ns p50 queue latency** (p50: 384 cycles) on Intel i7-13620H, with TSC-based measurement and log2 latency histograms.
- Designed for zero consumer stalls and minimal measurement overhead, supporting deep-queue stress tests and percentile latency tracking.
- Provided reproducible build scripts and clear documentation for low-latency, cache-efficient queue design.

## Competitions

---

#### IMC Prosperity 4 Trading Competition

2026

- Achieved Country Rank 130/1000 teams (Top 13%)
- Achieved Global Rank 750/18000 teams (Top 4%)

#### QuantStorm IIT Bombay

2026

- Achieved Rank 36/2000 participants (Top 1.8%)

## Education

---

#### Bachelor of Technology in Computer Science and Engineering

*Specialization: Cyber Security, Macro and Micro Economics*

#### Certifications

- NVIDIA - MODERN CUDA C++
- Google - Foundations of Cybersecurity
- ISC2 - Security Principles
- IBM - Introduction to Cybersecurity Tools and Attacks
- University of California San Diego - Data Structures
- IBM - Databases and SQL for Data Science with Python